Module Code and Title: SWDVC301 CONDUCT VERSION CONTROL

Learning Outcome 1: Setup repository
Learning Outcome 2: Manipulate files
Learning Outcome 3: Ship codes

Learning Outcome 1: Setup repository

Indicative content 1. Introduction to version control

1.1. Definition of general key terms

1. Repository

A **Git repository** tracks and saves the history of all changes made to the files in a Git project. It saves this data in a directory called **.git**, also known as the repository folder.

Git uses a version control system to track all changes made to the project and save them in the repository. Users can then delete or copy existing repositories or create new ones for ongoing projects.

Types of Git Repository

There are two types of Git repositories, based on user permissions:

Bare Repositories

Software development teams use **bare repositories** to share changes made by team members. Individual users aren't allowed to modify or create new versions of the repository.

Non-Bare Repositories

With **non-bare repositories**, users can modify the existing repository and create new versions. By default, the cloning process creates a non-bare repository.

2. Version control

Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later. Version control, also known as source control, is the practice of tracking and managing changes to software code. **Version control systems** are software tools that help software teams manage changes to source code over time.

If you are a graphic or web designer and want to keep every version of an image or layout (which you would most certainly want to), a Version Control System (VCS) is a very wise thing to use. It allows you to revert selected files back to a previous state, revert the entire project back to a previous state, compare changes over time, see who last modified something that might be causing a problem, who introduced an issue and when, and more. Using a VCS also generally means that if you screw things up or lose files, you can easily recover. In addition, you get all this for very little overhead.

3. Git

Git is the most commonly used version control system.

Git tracks the changes you make to files, so you have a record of what has been done, and you can revert to specific versions should you ever need to.

Git also makes collaboration easier, allowing changes by multiple people to all be merged into one source.

So regardless of whether you write code that only you will see, or work as part of a team, Git will be useful for you.

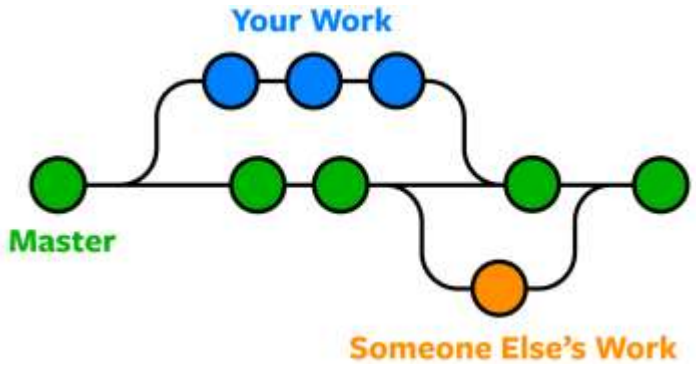


Figure 1 Git

4. GitHub:

GitHub is a code hosting platform for version control and collaboration. It lets you and others work together on projects from anywhere.

GitHub is *an online software development platform used for storing, tracking, and collaborating on software projects.*

GitHub is *a Git repository hosting service that provides a web-based graphical interface.*

5. Terminal:

The terminal *is an interface that lets you access the command line(Bash and CMD).*

Bash (Bourne Again Shell) is the free and enhanced version of the Bourne shell distributed with Linux and GNU operating systems. Bash is similar to the original, but has added features such as command-line editing.

CMD is an acronym for Command. Command prompt, or CMD, is the command-line interpreter of Windows operating systems. It is similar to Command.com used in DOS and Windows 9x systems called "MS-DOS Prompt". It is analogous to Unix Shells used on Unix like system.

The command prompt is a native application of the Windows operating system and gives the user an option to perform operations using commands.

1.2. Types of version control

- **Centralized version control**

With centralized version control systems, you have a single "central" copy of your project on a server and commit your changes to this central copy.

A centralized version control system offers software development teams a way to collaborate using a central server. In a centralized version control system (CVCS), a server acts as the main repository which stores every version of code.

You pull the files that you need, but you never have a full copy of your project locally. Some of the most common version control systems are centralized, including Subversion (SVN) and perforce.

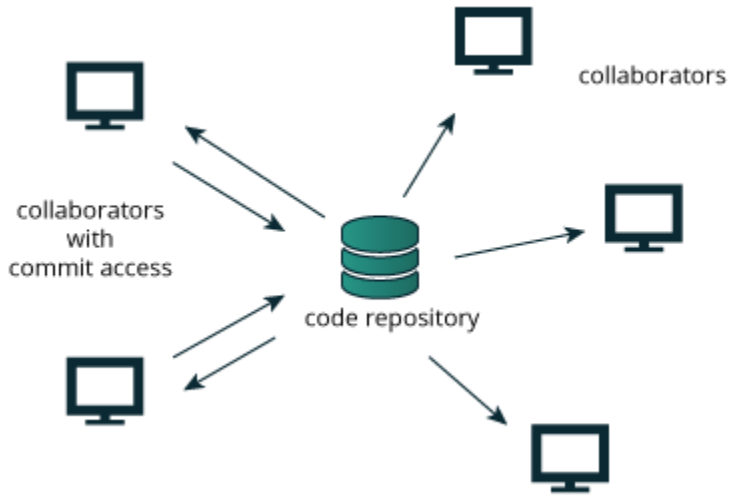


Figure 2 Centralised Version Control

- **Distributed version control**

With distributed version control systems (DVCS), you don't rely on a central server to store all the versions of a project's files. Instead, you clone a copy of a repository locally so that you have the full history of the project. Two common distributed version control systems are **Git and Mercurial**.

While you don't have to have a central repository for your files, you may want one "central" place to keep your code so that you can share and collaborate on your project with others. That's where Bitbucket comes in. Keep a copy of your code in a repository on Bitbucket so that you and your teammates can use Git or Mercurial locally and to push and pull code.

A distributed version control system (DVCS) is a type of version control where the complete codebase — including its full version history — is mirrored on every developer's computer. It's abbreviated DVCS. Changes to files are tracked between computers.

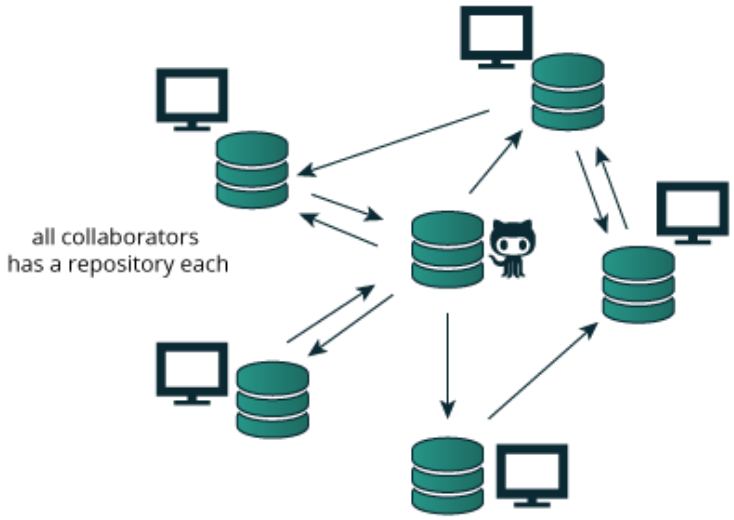


Figure 3 Distributed Version Control

- **Local version control**

A local version control system is a local database located on your local computer, in which every file change is stored as a patch.

Well Known version control system

1. Git
2. CVS (Concurrent Version System)
3. Mercurial
4. SVN (subversion)
5. GitLab
6. AWS Code Commit
7. Perforce
8. Beanstalk
9. Team Foundation Server
10. Bitbucket

Benefits of Version control

- ✓ Help in managing and protecting the source code
- ✓ Keep track of all the modifications made to the code
- ✓ Comparing earlier versions of the code
- ✓ Supports developer's workflow and not any rigid way of working

1.3. Indicative content : Description of git

3.1. Git basic concepts

1. Branch

A branch represents an independent line of development. Branches serve as an abstraction for the edit/stage/commit process.

2. Clone

The git clone command is used to create a copy of a specific repository or branch within a repository.

3. Pull

The git pull command is used to fetch and download content from a remote repository and immediately update the local repository to match that content.

4. Push

The git push command is **used to upload local repository content to a remote repository**. Pushing is how you transfer commits from your local repository to a remote repo. It's the counterpart to git fetch, but whereas fetching imports commits to local branches, pushing exports commits to remote branches.

5. Commit

It is used to record the changes in the repository. It is the next command after the git add. Every commit contains the index data and the commit message

6. Init

The git init command **creates a new Git repository**. It can be used to convert an existing, unversioned project to a Git repository or initialize a new, empty repository. Most other Git commands are not available outside of an initialized repository, so this is usually the first command you'll run in a new project.

The git init command is the first command that you will run on Git. The git init command is used to create a new blank repository.

3.2. Git architecture

Many VCS's use a two-tier architecture i.e a repository and a working copy. Git uses three-tier architecture i.e a working directory, staging area and local repository. The three stages of git can store different (or the same) states of the same code in each stage.

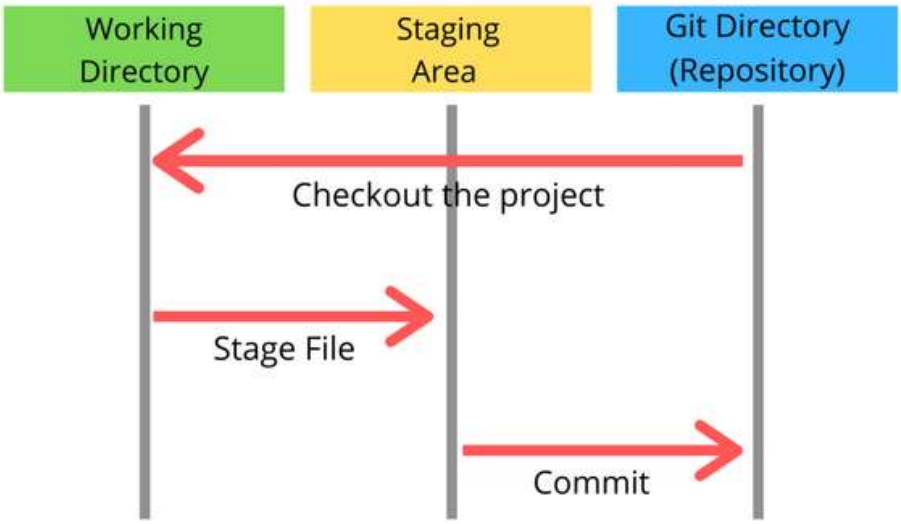


Figure 4 Git Architecture

3.3. Git workflow

A Git workflow is **a recipe or recommendation for how to use Git to accomplish work in a consistent and productive manner**. Git workflows encourage developers and DevOps teams to leverage Git effectively and consistently. Git offers a lot of flexibility in how users manage changes.

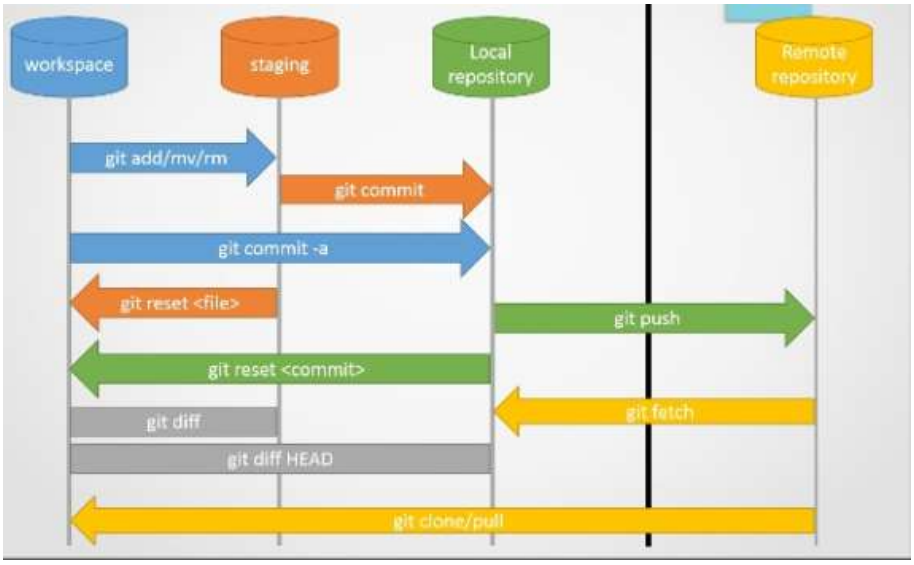


Figure 5 Git workflow in detail

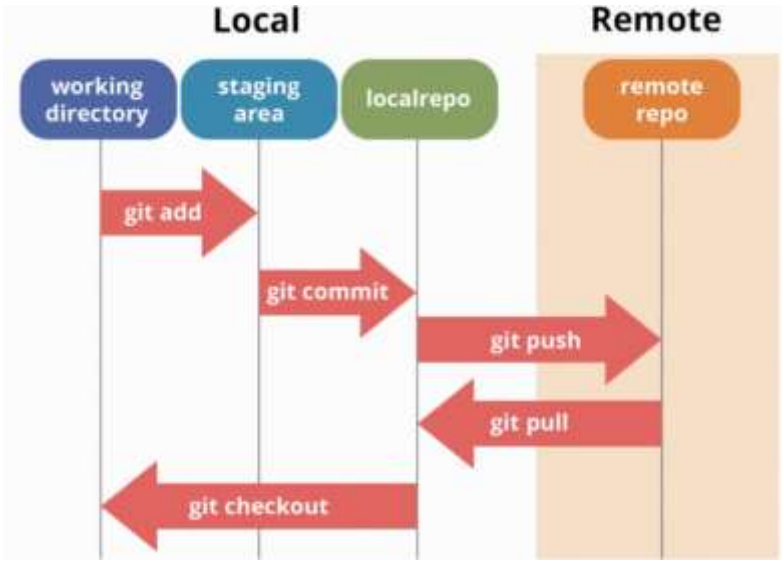


Figure 6 Git workflow simplified

3.4. INITIALISATION OF GIT

3.4.1. TERMINAL BASIC COMMANDS

git init

This command turns a directory into an empty Git repository. This is the first step in creating a repository. After running git init, adding and committing files/directories is possible.

git add

Adds files in the to the staging area for Git. Before a file is available to commit to a repository, the file needs to be added to the Git index (staging area). There are a few different ways to use git add, by adding entire directories, specific files, or all unstaged files.

git commit

Record the changes made to the files to a local repository. For easy reference, each commit has a unique ID.

git status

This command returns the current state of the repository.

git status will return the current working branch. If a file is in the staging area, but not committed, it shows with *git status*. Or, if there are no changes it'll return *nothing to commit, working directory clean.*

git config

With Git, there are many configurations and settings possible. *git config* is how to assign these settings. Two important settings are user.name and user.email. These values set what email address and name commits will be from on a local computer. With *git config*, a --*global* flag is used to write the settings to all repositories on a computer. Without a --*global* flag settings will only apply to the current repository that you are currently in.

git branch

To determine what branch the local repository is on, add a new branch, or delete a branch.

git checkout

To start working in a different branch, use *git checkout* to switch branches.

git merge

Integrate branches together. *git merge* combines the changes from one branch to another branch. For example, merge the changes made in a staging branch into the stable branch.

git remote

To connect a local repository with a remote repository. A remote repository can have a name set to avoid having to remember the URL of the repository.

git clone

To create a local working copy of an existing remote repository, use *git clone* to copy and download the repository to a computer. Cloning is the equivalent of *git init* when working with a remote repository. Git will create a directory locally with all files and repository history.

git pull

To get the latest version of a repository run *git pull*. This pulls the changes from the remote repository to the local computer.

git push

Sends local commits to the remote repository. *git push* requires two parameters: the remote repository and the branch that the push is for.

git log

To show the chronological commit history for a repository. This helps give context and history for a repository. *git log* is available immediately on a recently cloned repository to see history.

git rm

Remove files or directories from the working index (staging area). With *git rm*, there are two options to keep in mind: force and cached. Running the command with force deletes the file. The cached command removes the file from the working index. When removing an entire directory, a recursive command is necessary.

3.4.2. INSTALLATION OF GIT SETUP

To use Git, you have to install it on your computer. Even if you have already installed Git, it's probably a good idea to upgrade it to the latest version. You can either install it as a package or via another installer or download it from its official site.

Now the question arises that how to download the Git installer package. Below is the stepwise installation process that helps you to download and install the Git.

Step1

To download the Git installer, visit the Git's official site and go to download page. The link for the download page is https://git-scm.com/downloads. The page looks like as

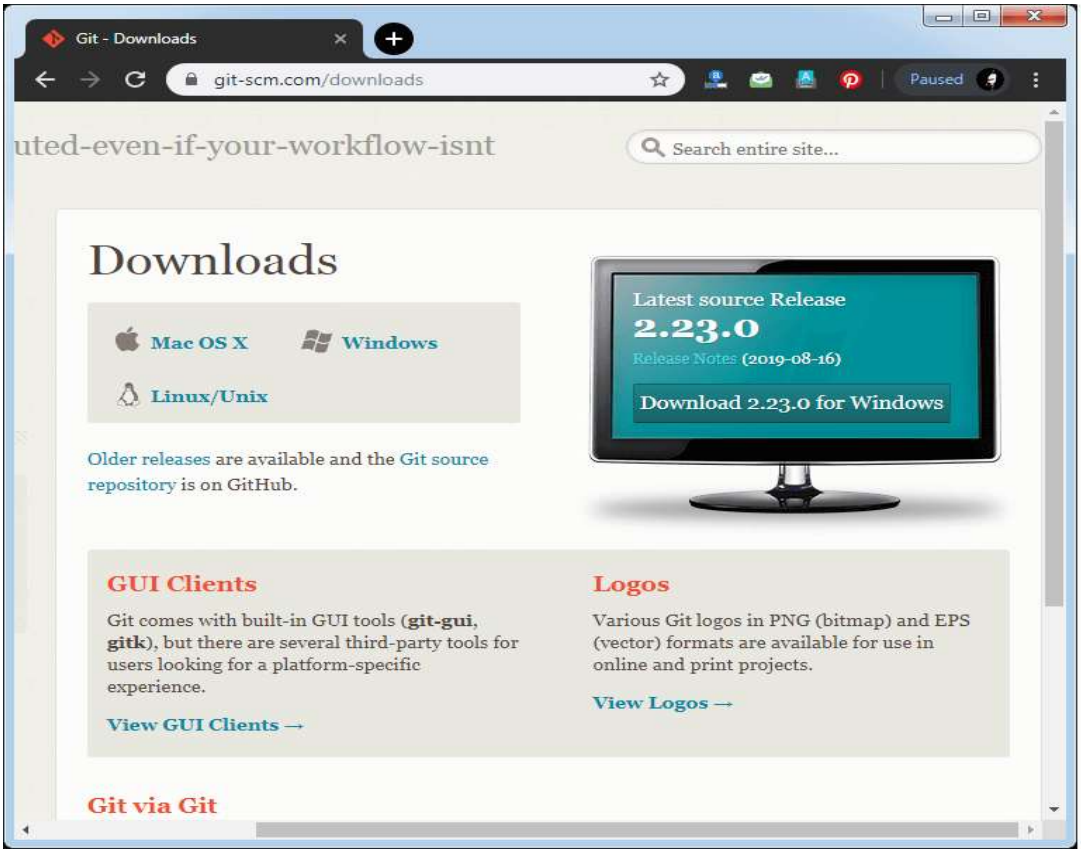


Figure 7 Installation of git step 1

Click on the package given on the page as **download 2.23.0 for windows**. The download will start after selecting the package.

Now, the Git installer package has been downloaded.

Step2

Click on the downloaded installer file and select **yes** to continue. After the selecting **yes** the installation begins, and the screen will look like as

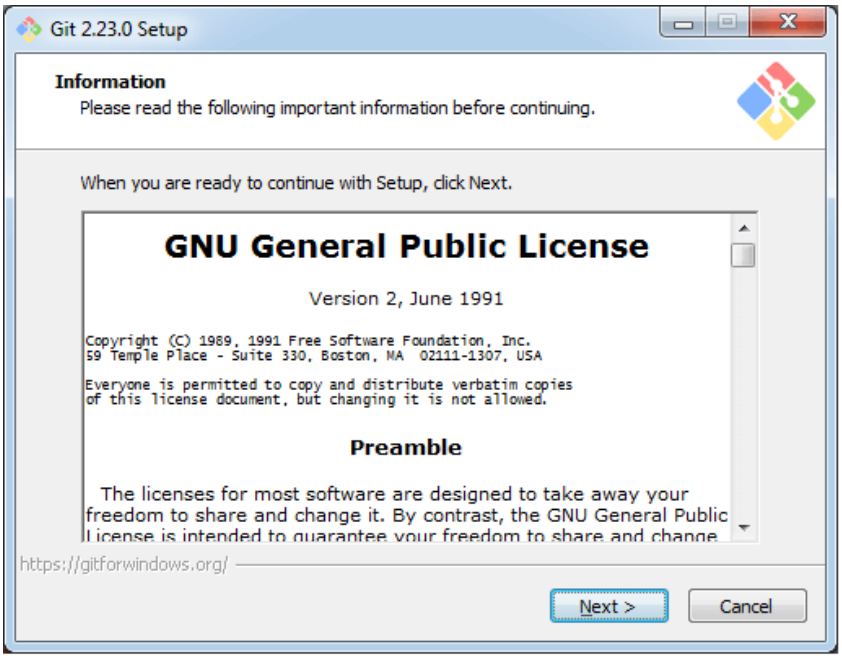
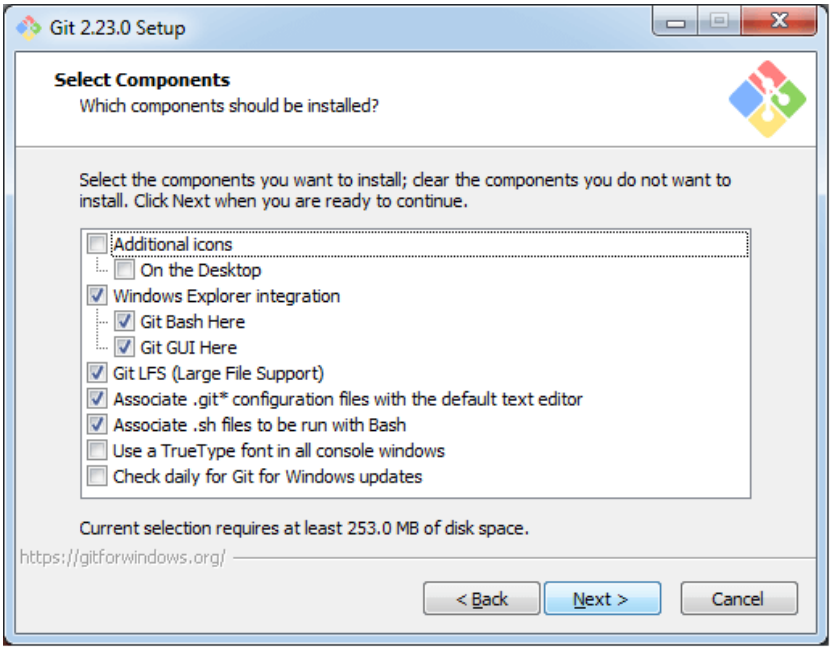


Figure 8 Install Git Step 2

Click on **next** to continue.

Step3

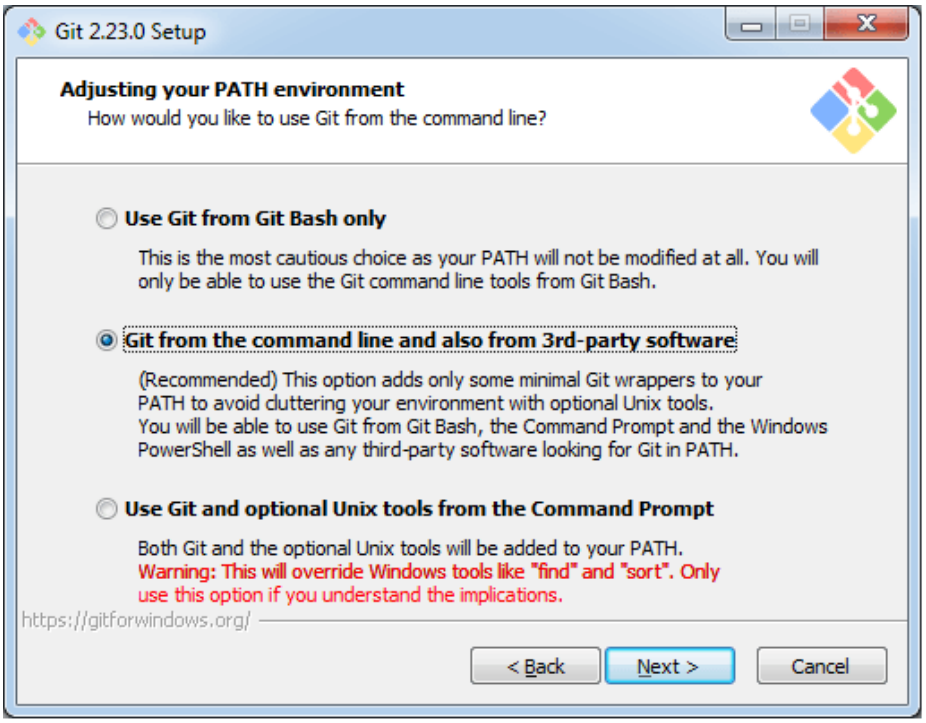
Default components are automatically selected in this step. You can also choose your required part.



Click next to continue.

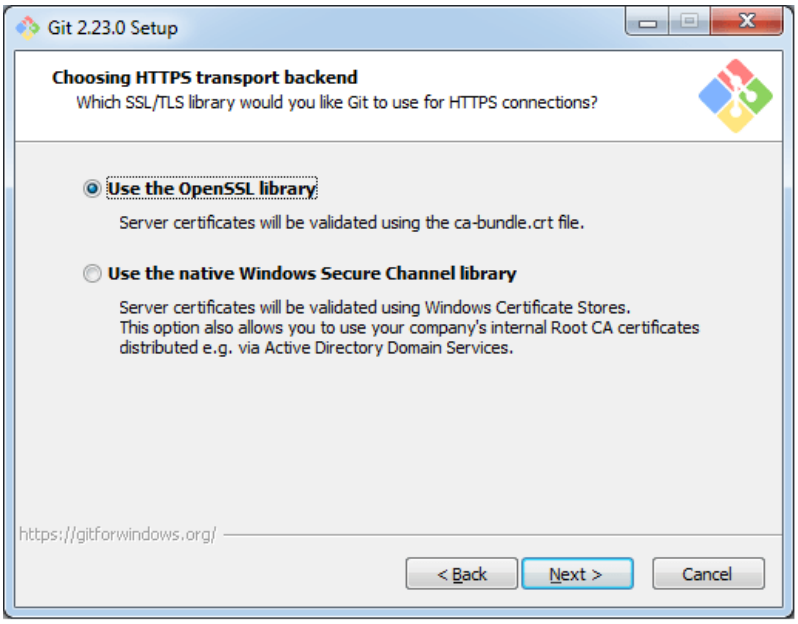
Step4

The default Git command-line options are selected automatically. You can choose your preferred choice. Click **next** to continue.



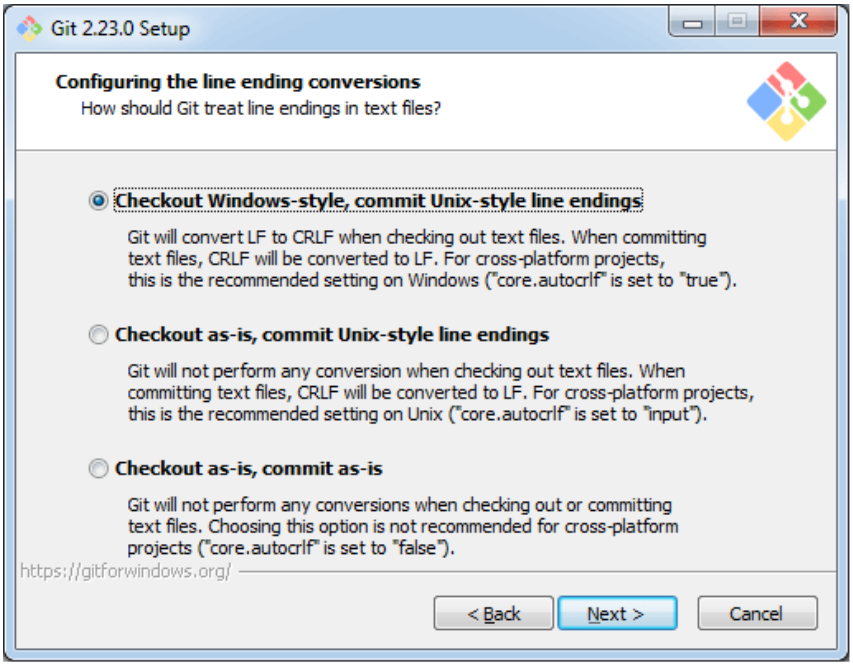
Step5

The default transport backend options are selected in this step. Click **next** to continue.



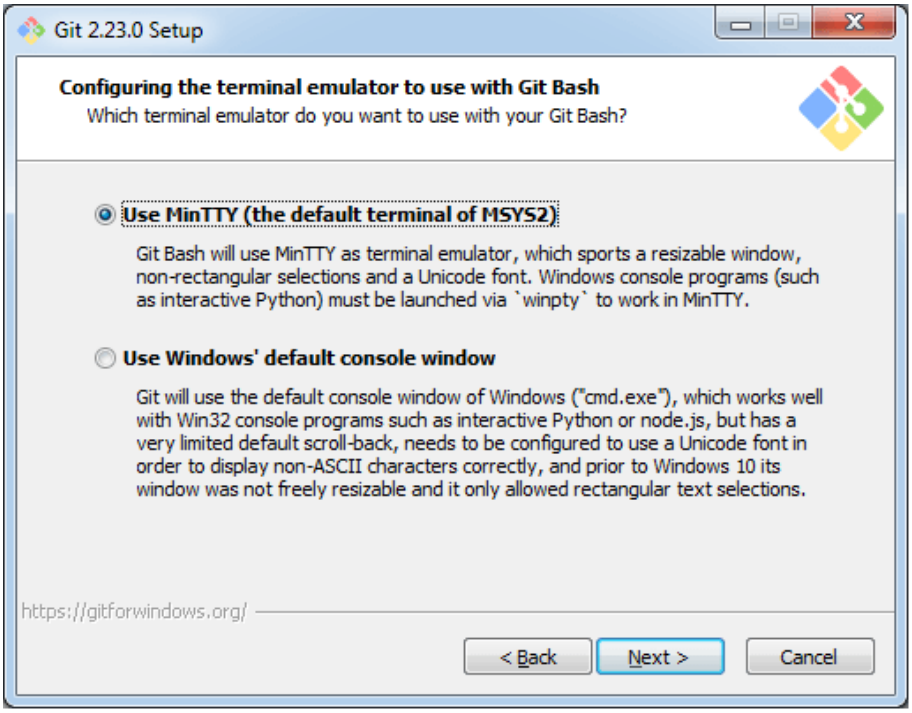
Step6

Select your required line ending option and click next to continue.



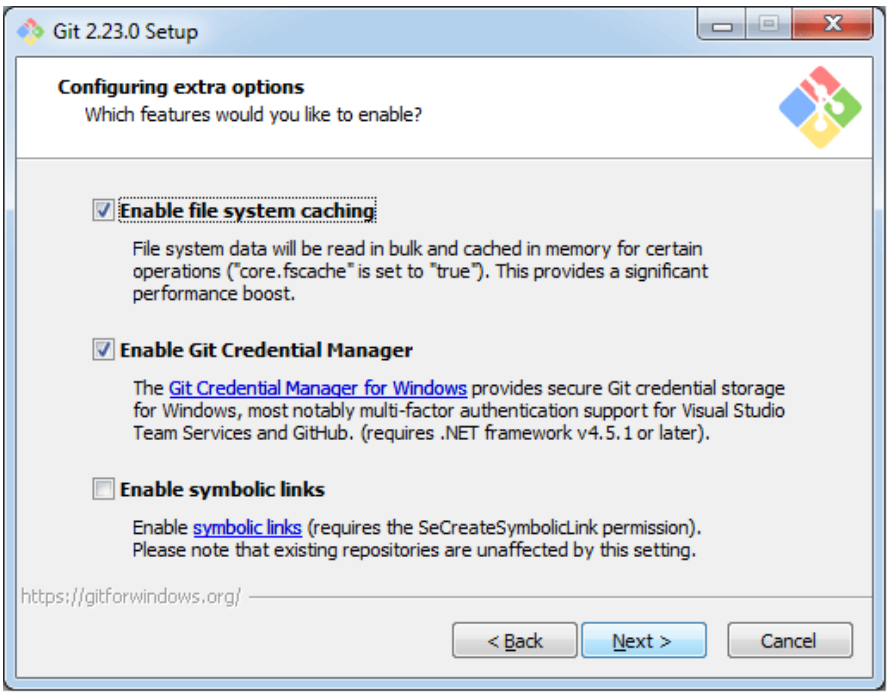
Step7

Select preferred terminal emulator clicks on the **next** to continue.



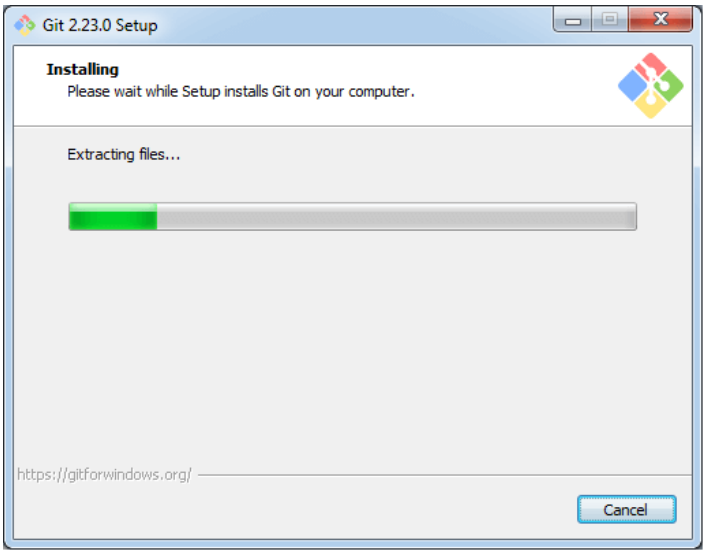
Step8

This is the last step that provides some extra features like system caching, credential management and symbolic link. Select the required features and click on the **next** option.



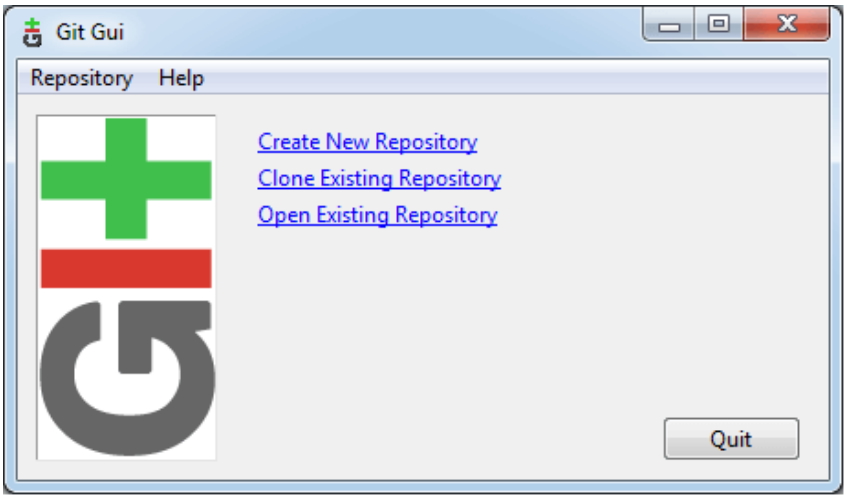
Step9

The files are being extracted in this step.



Therefore, The Git installation is completed. Now you can access the **Git Gui** and **Git Bash**.

The **Git Gui** looks like as



It facilitates with three features.

- Create New Repository
- Clone Existing Repository
- Open Existing Repository

The **Git Bash** looks like as

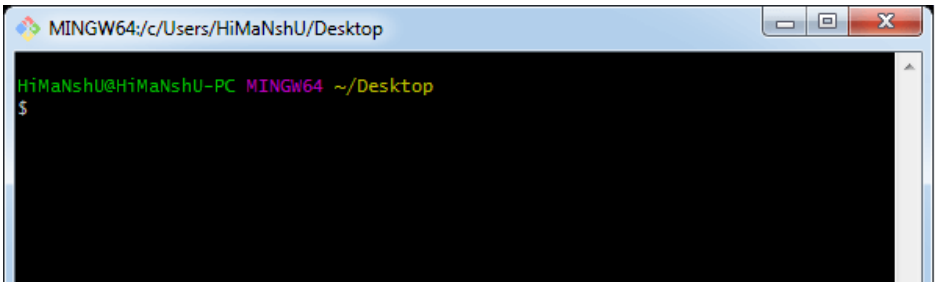


Figure 9 GitBash GUI

3.5. CONFIGURE GIT

3.5.1. Git init command

This command is used to create a local repository.

Syntax

$ git init Demo

The init command will initialize an empty repository. See the below screenshot.

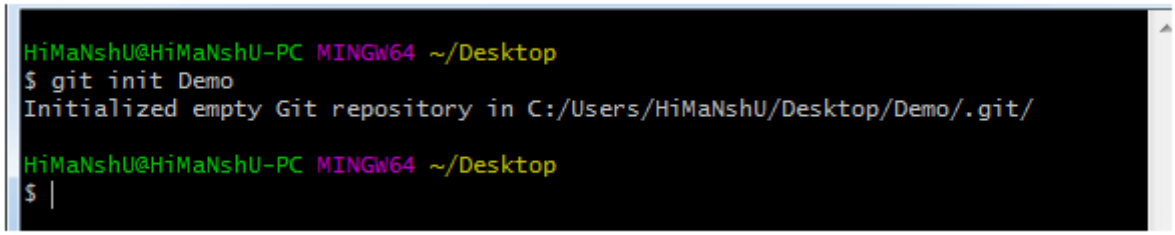


Figure 10 Git init example

3.5.2. Git config command

This command configures the user. The Git config command is the first and necessary command used on the Git command line. This command sets the author name and email address to be used with your commits. Git config is also used in other scenarios.

$ git config --global user.name "ImDwivedi1"

$ git config --global user.email "Himanshudubey481@gmail.com"

3.5.3. GIT - -VERSION COMMAND

Open the *command* prompt "terminal" and type *git version* to verify *Git* was installed.

The difference between GIT and SVN is

a) Git is less preferred for handling extremely large files or frequently changing binary files while SVN can handle multiple projects stored in the same repository.

b) GIT does not support 'commits' across multiple branches or tags. Subversion allows the creation of folders at any location in the repository layout.

c) Gits are unchangeable, while Subversion allows committers to treat a tag as a branch and to create multiple revisions under a tag root.

What are the advantages of using GIT?

a) Data redundancy and replication

b) High availability

c) Only one.git directory per repository

d) Superior disk utilization and network performance

e) Collaboration friendly

f) Any sort of projects can use GIT

Why GIT better than Subversion?

GIT is an open-source version control system; it will allow you to run 'versions' of a project, which show the changes that were made to the code overtime also it allows you keep the backtrack if necessary and undo those changes. Multiple developers can check out, and upload changes and each change can then be attributed to a specific developer.

Certainly! Here are some common questions and answers about version control:

Q: What is version control?

A: Version control is a system that helps track and manage changes to files and projects over time. It allows multiple people to collaborate on a project, keep track of changes, and easily revert back to previous versions if needed.

Q: Why is version control important?

A: Version control is important because it helps maintain a history of changes made to a project. It enables collaboration, allows for easy bug tracking and issue resolution, and provides a safety net to revert back to previous versions if something goes wrong.

Q: How does version control work?

A: Version control systems (VCS) track changes made to files and folders in a repository. They store each change as a separate version, allowing users to compare, merge, and manage different versions of files. Common version control systems include Git, Subversion (SVN), and Mercurial.

Q: What are the benefits of using version control?

A: Using version control offers several benefits, such as:

1. Collaboration: Multiple people can work on the same project simultaneously, making it easier to manage and merge changes.

2. History tracking: Version control systems keep a record of every change made to a project, allowing users to see who made the changes and when.

3. Branching and merging: Version control systems allow users to create branches to work on different features or experiments independently. These branches can later be merged back into the main project.

4. Rollbacks: If a mistake is made or something goes wrong, version control allows you to revert back to a previous working version of the project.

Q: What is the difference between centralized and distributed version control?

A: Centralized version control systems (e.g., SVN) have a central server that stores the repository, and users need to connect to that server to access and make changes to the project. Distributed version control systems (e.g., Git) create a local copy of the entire repository on each user's machine, allowing them to work offline and independently. Users can synchronize their changes with others by pushing and pulling from remote repositories.

3.6. Configure.git ignore file

Git Ignore

When sharing your code with others, there are often files or parts of your project, you do not want to share.

Examples

- log files
- temporary files
- hidden files
- personal files
- etc.

Git can specify which files or parts of your project should be ignored by Git using a .gitignore file.

Git will not track files and folders specified in .gitignore. However, the .gitignore file itself **is** tracked by Git.

Create .gitignore

To create a .gitignore file, go to the root of your local Git, and create it:

Example

```
touch .gitignore
```

Now open the file using a text editor.

We are just going to add two simple rules:

- Ignore any files with the .log extension
- Ignore everything in any directory named temp

Indicative content 4: Use of GitHub repository

4.1. Description of GitHub

GitHub is a web-based platform that provides a collaborative environment for software developers to work on projects. It serves as a version control system, allowing multiple developers to contribute to a project simultaneously and keep track of changes made to the codebase. GitHub also offers features like issue tracking, project management tools, and a marketplace for various software development tools and services. It has become a popular platform for open-source projects, enabling developers from around the world to collaborate and share their work.

4.2. To create an account on GitHub, you can follow these steps:

1. Open your web browser and go to the GitHub website (github.com).

2. On the homepage, click on the "Sign up" button located at the top right corner.

3. You will be directed to the account creation page. Here, you have two options:

- Sign up with your GitHub account by clicking on "Sign up with GitHub" if you already have a GitHub account.

- Alternatively, you can sign up with your email address by filling in the required information in the form provided.

4. If you choose to sign up with your email address, enter your preferred username, a valid email address, and a strong password.

5. Once you've filled in the necessary information, click on the "Create account" button.

6. GitHub may prompt you to verify your email address. Check your email inbox for a verification email from GitHub and follow the instructions to verify your account.

That's it! You have successfully created an account on GitHub. You can now start using GitHub to collaborate on projects, contribute to open-source software, and explore the vast community of developers.

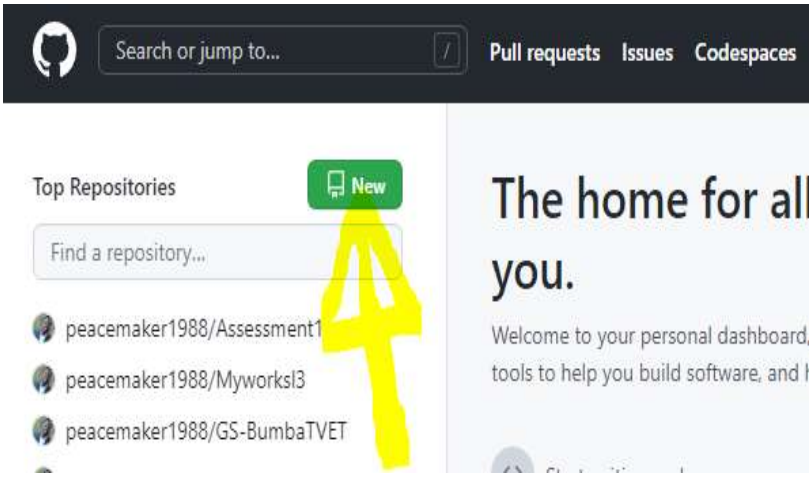
4.3. Create new remote repository

You have two options to create a Git repo. You can create one from the code in a folder on a computer, or clone one from an existing repo. If working with code that's just on the local computer, create a local repo using the code in that folder.

But most of the time the code is already shared in a Git repo, so cloning the existing repo to the local computer is the recommended way to go.

Once creating a remote repository, you can use GitHub interface by following those steps.

Step 1



Or click on the icon located near the profile picture as shown on that image.

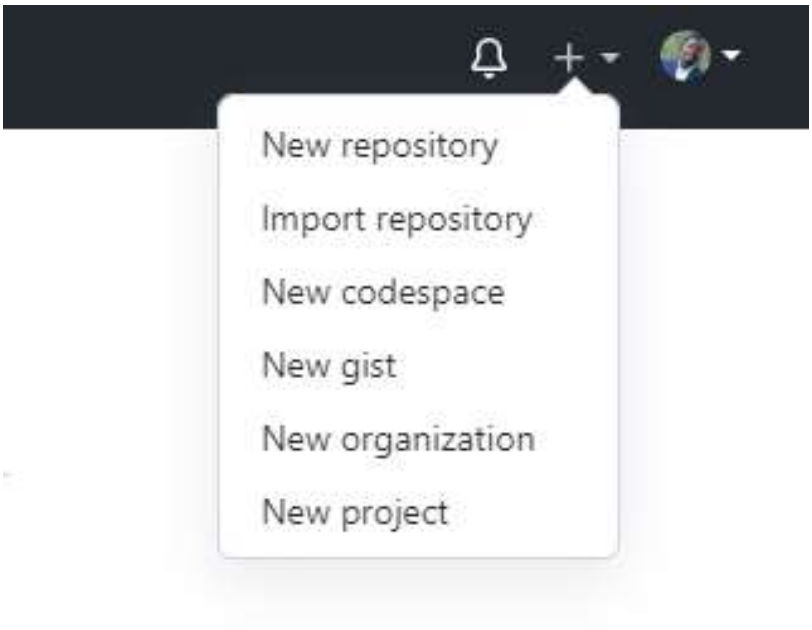
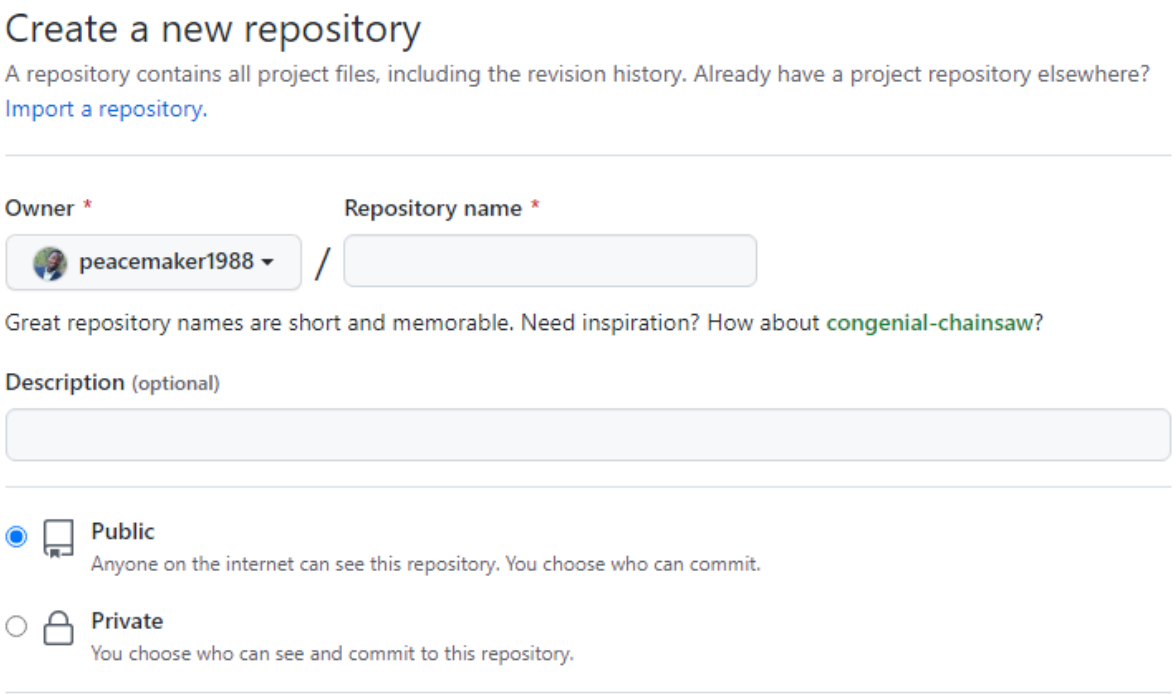


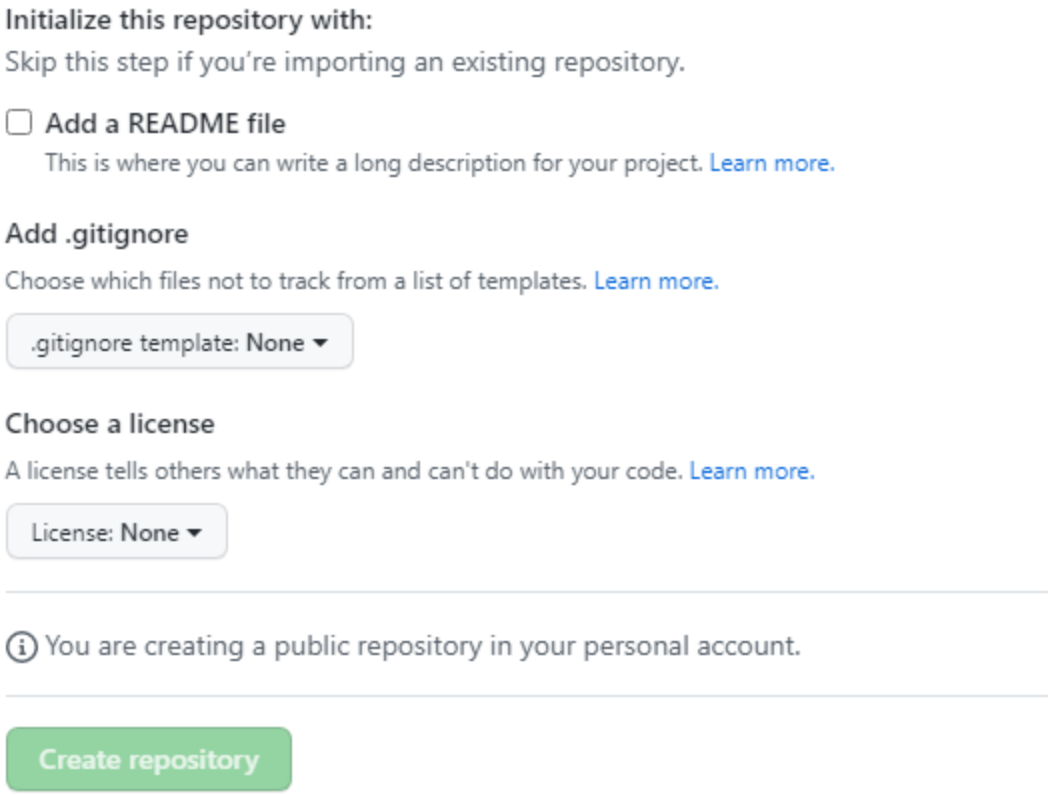
Figure: Creation of Remote repository

Step 2

Fill required information such as Repository name and Description as shown on the next figure and click on create repository.



New Repository Name and Description



4.4. Apply git commands related to repository

Here's a brief description of some commonly used Git commands related to repositories:

1.**`git clone`:**

- To create a local copy of an existing repository from a remote server, use the command `git clone <repository URL>`.

- Replace `<repository URL>` with the URL of the repository you want to clone.

- The `git clone` command is used to create a local copy of a remote repository.

- It allows you to download the entire repository, including all files, commit history, and branches, to your local machine.

- The command syntax is `git clone <repository URL>`, where `<repository URL>` is the URL of the remote repository you want to clone.

- For example, `git clone https://github.com/username/repository.git`.

- After executing the command, Git will create a new directory with the same name as the repository and copy all the repository files into it.

2. `git remote`:

- The `git remote` command is used to manage remote repositories linked to your local repository.

- It allows you to view, add, rename, or remove remote repositories.

- Some commonly used `git remote` commands are:

  - `git remote -v`: Lists all the remote repositories associated with your local repository, along with their URLs.

  - `git remote add <name> <URL>`: Adds a new remote repository with a specified name and URL.

  - `git remote rename <old-name> <new-name>`: Renames an existing remote repository.

  - `git remote remove <name>`: Removes the specified remote repository from your local repository.

3. **Initialize a Repository**:

- To create a new repository, navigate to the desired directory in your terminal or command prompt.

- Use the command `git init` to initialize a new Git repository in that directory.

4. **Add Files to the Repository**:

- Use the command `git add <file>` to stage changes for a specific file.

- Replace `<file>` with the name of the file you want to add.

- You can also use `git add .` to stage all changes in the current directory.

5. **Commit Changes**:

- After adding files, use the command `git commit -m "commit message"` to commit the changes.

- Replace `"commit message"` with a brief description of the changes you made.

6. **Push Changes to Remote Repository**:

- To upload your local commits to the remote repository, use the command `git push`.

- This will push your committed changes to the default branch of the remote repository.

7. **Pull Changes from Remote Repository**:

- To fetch and merge the latest changes from the remote repository to your local repository, use the command `git pull`.

- This will update your local repository with the latest changes made by others

8. **Check Repository Status**:

- You can use the command `git status` to check the current status of your repository.

- It will show you which files are modified, staged, or untracked.

9. **View Commit History**:

- Use the command `git log` to view the commit history of your repository.

- This will display a list of commits, including the commit message, author, date, and commit hash.

Learning outcome 2: Manipulate files

Indicative content 1: Definition of general key terms

- **Status:** refers to the current state or condition of files or changes within a repository.
- **Branch:** A branch is a parallel timeline of development that allows multiple versions of a project to coexist. It enables developers to work on different features or bug fixes independently, without interfering with each other's work.
- **Commit:** Committing is the act of saving changes to the repository. It creates a new version of the files, capturing the modifications made since the

last commit. Each commit is accompanied by a commit message to describe the changes made.

Indicative content 2: Add file change to git staging area

2.1. Operation on git status command

● **View new untracked file:** To view new untracked files in Git, you can use the `git status` command. When you run `git status`, Git will show you a list of untracked files under the "Untracked files" section. This way, you can easily see which files are new and not yet added to the staging area for the next commit.

● **View modified file:** To view modified files in Git, you can also use the `git status` command. When you run `git status`, Git will display a list of modified files under the "Changes not staged for commit" section.

● **View deleted file:** To view deleted files in Git, you can once again use the `git status` command.

2.2. Operation on git add command

● **Stage all files:** To stage all files in Git for the next commit you can use **git add .** This command adds all modified, new, and deleted files to the staging area. The dot **`.`** in the command represents the current directory, so it stages all changes in the current directory and its subdirectories. After running `git add .`, you can then proceed to commit the changes using **`git commit -m "Your commit message"`** to finalize the changes.

● **Stage a file:** To stage a specific file in Git for the next commit, you can use the **git add <file_name>**

Git add with the name of the file you want to stage. This command adds the specified file to the staging area, preparing it to be included in the next commit. After staging the file, you can proceed to commit the changes using **`git commit -m "Your commit message"`** to finalize the changes.

● **Stage folder:** Git add <folder_name>

2.3. Operation on git reset command

● **Unstage a file: git reset <file_name>** This command will remove the specified file from the staging area, but it will keep your changes in the working directory. If you want to unstage all files that have been staged, you can use **git reset**

● **Deleting and staging file/folder:**

To unstage a file:

- If you have staged a file and want to remove it from the staging area, you can use:

git reset HEAD <file_name>

or git restore --staged <file_name>

2.4. Operation on rm command

● **Remove and stage a file:**

- **git rm <file_name>** (to remove file from both staging and working directory)
- **Git rm -r .** (to remove all files from both staging and working directory)
- **Git rm --cached<file_name>** (remove files from only staging area)

● **Remove and stage a folder:**

- **git rm <folder_name>** (to remove folders from both staging and working directory)
- **Git rm -r .** (to remove all folders from both staging and working directory)
- **Git rm --cached<folder_name>** (remove folders from only staging area)

Indicative content 3: Commit File changes to git local repository

3.1. Best practice of creating a commit message

3.2. Operation on git commit command

● **Commit a file**

● **Edit commit message** (git commit -m" commit message")

3.3. Operation on git log command

● **To see simplified list of commits** (git log)

● **To see a list of commits with more detail** (git log)

Indicative content 3: Manage branches

3.1. Operations on branches

●Create branch

Git branch<branch-name>

Here are some additional points to remember:

- The new branch name should be descriptive and follow your repository's naming conventions.
- Ensure you have saved any unstaged changes before creating a new branch, as they might be lost.
- After creating a branch, you can use git branch to list all branches and git checkout <branch-name> to switch between them.

● List branch

There are two ways to create a new branch in Git:

1. Creating a new branch:

This method creates the branch but doesn't switch to it.

- **Open your terminal or command prompt.**
- **Navigate to the root directory of your Git repository.** You can use the cd command to do this.
- **Run the following command, replacing** <branch-name> **with the desired name for your new branch:**

git branch <branch-name>

2. Creating and switching to a new branch:

This method both creates the branch and switches to it, allowing you to start working on it immediately.

- **Run the following command, replacing** <branch-name> **with the desired name for your new branch:**

git checkout -b <branch-name>

Here are some additional points to remember:

- The new branch name should be descriptive and follow your repository's naming conventions.
- Ensure you have saved any unstaged changes before creating a new branch, as they might be lost.
- After creating a branch, you can use git branch to list all branches and git checkout <branch-name> to switch between them.

● Delete local and remote branch

Deleting a local and remote branch in Git involves two separate commands:

1. Deleting the local branch:

- Open your terminal or command prompt.
- Navigate to the root directory of your Git repository.
- Use the following command, replacing <branch-name> with the name of the local branch you want to delete:

git branch -d <branch-name>

Optionally, use the -D **flag for force deletion:**

git branch -D <branch-name>

- The -d flag (default) only allows deletion if the branch is merged into the current branch.
- The -D (force deletion) flag allows deletion even if the branch is not merged, but **use this with caution** as it can lead to data loss.

2. Deleting the remote branch (if applicable):

- **After deleting the local branch, if you want to remove the corresponding remote branch, use the following command:**

git push origin :<branch-name>

- Replace <branch-name> with the name of the branch you deleted locally.

Important points to remember:

- Be cautious when deleting branches, especially with the -D flag, as it's permanent.

- Ensure you have the proper permissions to delete remote branches.
- Consider alternatives like archiving branches instead of deleting them, especially if they contain valuable work history.

● Switch branch

You can switch branches in Git using the git checkout command. Here's how:

Switching to an existing branch:

1. Open your terminal or command prompt.
2. Navigate to the root directory of your Git repository. You can use the cd command to do this.
3. Use the following command to switch branches:

git checkout <branch-name>

- Replace <branch-name> with the name of the branch you want to switch to.

Creating and switching to a new branch:

1. Use the following command to create a new branch and switch to it simultaneously:

git checkout -b <branch-name>

- Replace <branch-name> with the desired name for your new branch.

Additional options:

- View a list of all branches:

git branch

- Switch back to the previous branch:

git checkout -

Things to remember:

- Ensure you have saved any unstaged changes before switching branches, as they might be lost.

- When switching to a detached HEAD state (using the commit hash), you won't be on any specific branch and won't be able to directly commit changes. Use git checkout to return to a proper branch.

● **Rename branch**

The process of renaming a branch can be done in two ways, depending on whether the branch is local or remote:

Renaming a local branch:

1. Open your terminal or command prompt.
2. Navigate to the root directory of your Git repository. You can use the cd command to do this.
3. Use the following command to rename the branch:

git branch -m <old-name> <new-name>

- Replace <old-name> with the current name of the branch.
- Replace <new-name> with the desired new name for the branch.

Renaming a remote branch:

1. Follow steps 1 and 2 from the "Renaming a local branch" section.
2. Delete the old remote branch:

git push origin :<old-name>

- Replace <old-name> with the current name of the branch.

3. Push the renamed local branch to the remote repository:

git push origin -u <new-name>

- Replace <new-name> with the new name you gave the branch in step 3 of the "Renaming a local branch" section.

Here are some additional things to keep in mind:

- Make sure you are on the correct branch before renaming it.
- The new branch name must be unique and follow the same naming conventions as other branches in your repository.

- If you are working on a team, it is a good practice to communicate with your team members before renaming a branch.